Advanced Programming 2025

Predicting Song Popularity on Spotify Using Audio Features and Machine Learning

Final Project Report

Clémence Sarah Choukroun
clemence.choukroun@unil.ch
Student ID: 21401799

January 10, 2026

Abstract

This project addresses the problem of predicting song popularity on Spotify using machine learning techniques. Song popularity is influenced by a complex combination of audio features, musical attributes, temporal effects, and external factors that are not fully observable, which makes accurate prediction a challenging task. To address this problem, an existing dataset of Spotify tracks is analyzed and prepared for modeling. The methodology involves standard preprocessing steps to ensure data quality and comparability across models, followed by the implementation of several regression-based machine learning approaches. The models considered include Linear Regression, Ridge Regression, Random Forest regression, and XGBoost, all trained and evaluated under identical conditions. Model performance is assessed using the coefficient of determination (R^2) and complementary error-based metrics, supported by diagnostic plots. The results indicate that overall predictive performance remains limited across all models, suggesting that the available audio and musical features alone are insufficient to fully explain song popularity. While tree-based models, particularly XGBoost, outperform linear approaches, the relatively low R^2 values highlight both the inherent difficulty of the prediction task and the limitations of the chosen features and target variable. The main contributions of this project include a clear and reproducible application of machine learning techniques to Spotify popularity prediction, as well as a comparison of several regression models using the same data preparation and evaluation framework.

Keywords: Data Science, Python, Machine Learning, Hit Song Science, Musice Information Retrieval, Spotify, Music Popularity Prediction

Contents

1 Introduction **3**

2 Literature Review / Related Work **3**

3 Methodology **4**
3.1 Data Description 4
3.2 Approach 5
3.3 Implementation 5

4 Results **6**
4.1 Experimental Setup 6
4.2 Performance Evaluation 7
4.3 Visualizations 7

5 Discussion **8**

6 Conclusion and Future Work **9**
6.1 Summary 9
6.2 Future Directions 9

References **11**

A Additional Figures **12**

B Helper tools **12**

C Code Repository **13**

1 Introduction

The rapid growth of music streaming platforms has profoundly transformed the way music is produced, distributed, and consumed. Services such as Spotify rely heavily on data-driven algorithms to personalize user experiences, organize playlists, and recommend new content. Song popularity plays a central role in these systems, as it influences both content visibility and recommendation dynamics. Music recommendation systems are primarily driven by aggregated user behavior, such as listening frequency and engagement patterns, which closely align with Spotify's notion of popularity. Moreover, Spotify defines popular tracks as being determined automatically based on all-time streaming counts, with adjustments for the recency of listening activity, highlighting popularity as a dynamic and quantitative algorithmic measure of collective user engagement.

Despite its importance, predicting song popularity remains a challenging task. Musical success depends on a complex combination of factors, including musical characteristics, listener preferences, and evolving consumption trends. While factors such as marketing strategies, artist reputation, and viral dynamics on social media platforms can significantly affect popularity, they are difficult to quantify and are often unavailable in publicly accessible datasets. Consequently, an open question remains as to whether song-level audio features and metadata alone can provide sufficient information to model and predict popularity.

The objective of this project is to investigate whether audio features and metadata provided by Spotify can be used to predict song popularity scores. Using a dataset of songs enriched with quantitative audio descriptors, this study applies several machine learning techniques to model the relationship between musical characteristics and popularity. In particular, the project aims to compare linear and non-linear models, evaluate their predictive performance, and identify which features contribute most to explaining variations in popularity. Additionally, the project seeks to assess the extent to which popularity can be inferred from musical content alone, while remaining consistent with Spotify's definition of popularity by explicitly accounting for temporal aspects of music consumption.

The remainder of this report is organized as follows. Section 2 reviews the relevant literature. Section 3 presents the data set, describes the feature engineering process, and describes the machine learning methods used. Section 4 reports the empirical results using evaluation metrics and visualizations. Section 5 discusses and interprets these findings. Finally, Section 6 concludes the report by summarizing the main results and presenting directions for future research.

2 Literature Review / Related Work

Research on song popularity prediction arises from the fields of music information retrieval (MIR) and recommender systems, where popularity is viewed as the outcome of interacting musical and social factors. This complexity leads to self-reinforcing dynamics that make popularity difficult to predict using song characteristics alone.

Within this literature, *Hit Song Science* (HSS) investigates whether a song's commercial success can be predicted prior to its release using audio-based features. As defined by Ni et al. (2015), HSS is grounded in the assumption that popular songs share musical characteristics that can be learned by machine learning models. However, early large-scale studies challenge this assumption. Pachet and Roy (2008) show that while certain subjective musical attributes can be learned from audio features, popularity itself cannot be reliably predicted using content-based features alone.

Later research suggests that content-based prediction becomes more informative when combined with temporal modeling. Lee et al. (2015) demonstrate that audio-derived features related to musical complexity, together with early-stage popularity indicators, can be used to predict long-term popularity patterns in music charts. Their results indicate that musical content con-

tributes meaningfully to popularity prediction when temporal dynamics are taken into account.

Machine learning methods play a central role in these approaches. Linear models are commonly used as interpretable baselines, while non-linear models such as tree-based and boosting algorithms better capture complex relationships between musical attributes and popularity. Across this literature, audio features and metadata are typically obtained from the Spotify API, which provides large-scale and standardized song-level information. Measures of popularity, however, are derived using different sources depending on data availability. For instance, Nijkamp (2018) relies on streaming counts obtained from external sources to model song success, while Berger (2017) uses Spotify's proprietary popularity metric in a content-based machine learning framework. Building on this body of work, the present study focuses on predicting Spotify song popularity using audio features and temporal metadata, while deliberately excluding external social or promotional signals.

3 Methodology

3.1 Data Description

The dataset used in this project was obtained from Kaggle and contains information on approximately 30,000 songs available on Spotify. The data were originally collected using the Spotify Web API and include standardized song-level metadata, audio features, and a popularity measure. As the popularity values provided in the original dataset were outdated, they were updated at the time of analysis using the Spotify Web API through the `spotipy` library, while all other variables remained unchanged.

Each observation in the dataset corresponds to a single song. In its initial form, the dataset consists of approximately 30,000 observations described by 23 variables. Based on the constructed song age variable, the dataset covers songs released over a broad temporal range, with song ages varying between 5 and 70 years. The distribution of popularity values is highly imbalanced, with a large proportion of songs exhibiting low popularity scores, approximately 40% of tracks have a popularity value of zero, while only a small fraction achieve high popularity, with around 2% of songs scoring above 80.

The final dataset used for analysis contains a combination of audio features, categorical attributes, and a popularity outcome. Audio features describe musical characteristics and include danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo, and song duration. In addition to audio features, the dataset includes information related to musical genre, musical key, and mode, which appear in the dataset as numerical indicator variables. A song age variable is also included to capture temporal aspects related to popularity. Columns containing identifiers and descriptive metadata, such as song, artist, album, and playlist information, were excluded from the final dataset used for analysis. The target variable is the updated Spotify popularity score, which ranges from 0 to 100 and reflects relative song popularity on the platform.

Overall, the dataset is of good quality, as the features are automatically generated by Spotify. The exploratory analysis highlights characteristics that are relevant for modeling: the popularity variable exhibits a strong skew, and no single audio feature shows a strong association with popularity. This suggests that popularity cannot be explained by individual musical attributes alone and may depend on a combination of features, as well as on factors not captured in the dataset. These observations motivate the use of multivariate modeling approaches in subsequent sections.

Additional figures illustrating the distribution of popularity scores and feature correlations are provided in the Appendix.

3.2 Approach

The objective of this project is to predict Spotify song popularity using supervised learning methods based on audio features and song-level metadata. Since the target variable is a continuous popularity score, the task is formulated as a regression problem. Several models with different modeling assumptions are considered in order to assess how various approaches capture the relationship between musical characteristics and popularity.

Linear regression is used as a baseline model due to its simplicity and interpretability, providing a reference for how much variation in popularity can be explained through linear relationships between features. Ridge regression is included to mitigate the effects of correlated audio features and to evaluate whether regularization leads to more robust predictions than standard linear regression. In addition to linear models, tree-based ensemble methods are considered to account for potential non-linear relationships and interactions between musical attributes. Random Forest and XGBoost are particularly well suited to this task because they can capture non-linear effects and interactions between musical attributes, which are likely to influence popularity given the absence of strong linear relationships observed in the data.

Before model estimation, the dataset undergoes several preprocessing steps to ensure data quality and comparability across models. Duplicate observations and missing values are removed to avoid inconsistencies during model training. Songs with a popularity score equal to zero are also excluded, as they typically indicate negligible engagement rather than informative variation in popularity. Including such tracks would strongly skew the target distribution and inflate prediction errors, while contributing little to explaining differences in popularity among actively streamed songs. Categorical variables related to musical genre, musical key, and mode are converted into numerical indicator variables, and continuous features are standardized to ensure comparable scales across variables. A song age variable is constructed from the release year to capture temporal effects related to popularity. All preprocessing steps are applied consistently across models to ensure a fair comparison of predictive performance. After preprocessing, the final dataset contains 16'050 unique songs, which remains a substantial sample size for model training and evaluation.

Model performance is evaluated using a train–test split, where the models are trained on one part of the data and tested on a separate set of songs. Evaluation relies on standard regression metrics, including the coefficient of determination (R^2), as well as mean absolute error (MAE) and root mean squared error (RMSE), which measure how far the predicted popularity scores deviate from the observed values.

3.3 Implementation

The project is implemented in Python and relies on standard libraries for data processing, machine learning, and visualization. Data manipulation and cleaning are performed using *pandas* and *NumPy*. Updates of the popularity variable are obtained through calls to the Spotify Web API using the *spotipy* library. Model training and evaluation are conducted using *scikit-learn* for linear regression, Ridge regression, and Random Forest models, while *XGBoost* is used for gradient boosting. Visualizations are generated using *matplotlib*.

The implementation is divided into two main phases. First, exploratory analysis, popularity updating, and data cleaning are carried out in Jupyter notebooks. During this phase, the raw dataset is cleaned and enriched, and the resulting dataset is saved for subsequent use. This step produces a fixed and consistent input dataset for the modeling stage.

In the second phase, the cleaned data are used as input for the modeling pipeline, which is executed through a central script. This script sequentially calls the data loading, model training, evaluation, and plotting components. The cleaned dataset is used to construct the feature matrix and target variable required for model estimation. Regression models are then trained using this data, and their performance is assessed through the computation of standard

evaluation metrics and the generation of comparison plots. Executing these steps through a single entry point ensures that all models are trained and evaluated under identical conditions. An illustrative code excerpt is provided below to highlight representative preprocessing steps.

```
# Remove duplicate observations
df = df.drop_duplicates()

# Remove missing values
df = df.dropna()

# Remove songs with zero popularity
df = df[df["updated_popularity"] > 0]

# Convert release date to datetime
df["track_album_release_date"] = pd.to_datetime(
    df["track_album_release_date"], errors="coerce"
)

# Create song age feature (in years)
df["song_age"] = (
    pd.Timestamp.today() - df["track_album_release_date"]
).dt.days / 365

# Drop original release date
df = df.drop(columns=["track_album_release_date"])

# Save cleaned dataset for modeling
df.to_csv("data/cleaned_spotify_data.csv", index=False)
```

Listing 1: Illustrative data cleaning and feature construction steps

4 Results

This section presents the empirical results of the Spotify popularity prediction task. Model performance is evaluated using quantitative metrics reported in a dedicated results table, complemented by visualizations that provide additional insight into prediction quality and error behavior.

4.1 Experimental Setup

All experiments were conducted on a standard personal computer. The models were implemented in Python and coded in Visual Studio Code. Linear and ensemble models were trained using the *scikit-learn* library, while gradient boosting was implemented using *XGBoost*.

Four regression models were evaluated: Linear Regression, Ridge Regression, Random Forest, and XGBoost. Linear Regression was trained using default parameters. Ridge Regression was evaluated using regularization parameters $\alpha = 0.5$, $\alpha = 1.0$, and $\alpha = 1.5$. No meaningful differences in R^2 or error metrics were observed across these values, and $\alpha = 1.0$ was retained.

The Random Forest model was configured with 200 estimators. Increasing the number of trees to 300 was also tested but did not result in any improvement in predictive performance while increasing training time. A fixed random seed (`random_state = 42`) was used for both Random Forest and XGBoost to ensure consistency across results.

For XGBoost, several values for the number of estimators were tested. The highest R^2 on the test set was obtained with 100 estimators, while additional trees led to slightly lower performance. The learning rate was set to 0.05, and a subsampling ratio of 0.8 was used, meaning that 80% of the training observations were randomly sampled at each boosting iteration.

4.2 Performance Evaluation

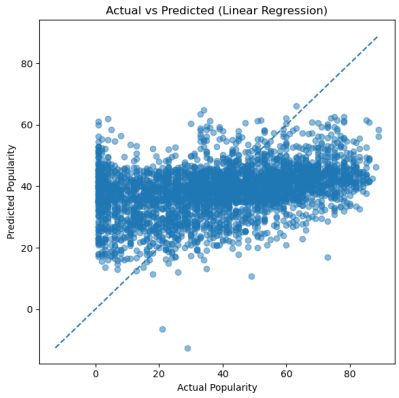
Table 1: Model Performance Metrics

Model	R^2	**RMSE**	**MAE**
Linear Regression	0.13	21.31	17.64
Ridge Regression	0.13	21.31	17.64
Random Forest	0.18	20.78	16.74
XGBoost	0.20	20.54	16.73

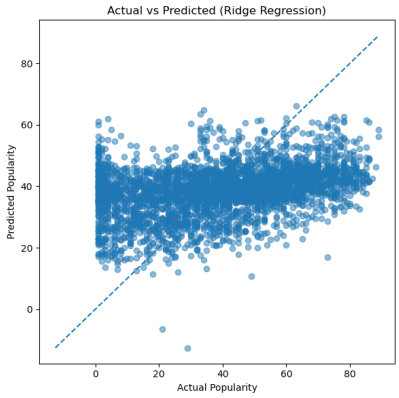
While the reported R^2 values are relatively low, this outcome is not unexpected given the complexity of the target variable. Low R^2 values indicate that audio features explain only a limited share of the variability in popularity, reflecting the strong influence of external factors such as marketing exposure, playlist placement, and social dynamics that are not captured in the dataset.

Error-based metrics such as RMSE and MAE provide a complementary view of model performance. MAE measures the average absolute difference between predicted and true popularity scores on the 0-100 scale, while RMSE is the square root of the mean squared prediction error and therefore penalizes large errors more strongly. RMSE values around 20 indicate substantial uncertainty in individual predictions on the 0-100 popularity scale. Nevertheless, consistent reductions in RMSE and MAE across models still reflect meaningful relative improvements in prediction accuracy.

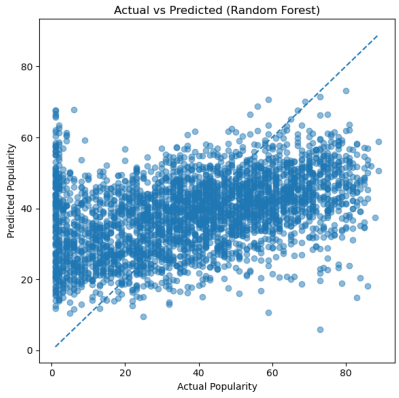
4.3 Visualizations



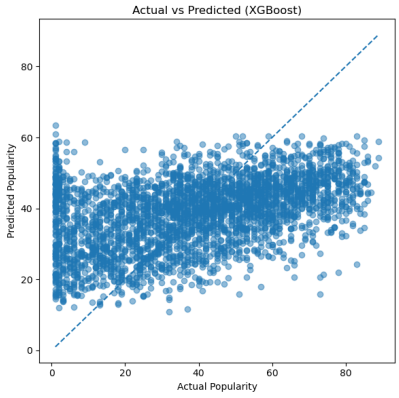
(a) Linear Regression



(b) Ridge Regression



(c) Random Forest



(d) XGBoost

Figure 1: Actual versus predicted popularity for all evaluated models.

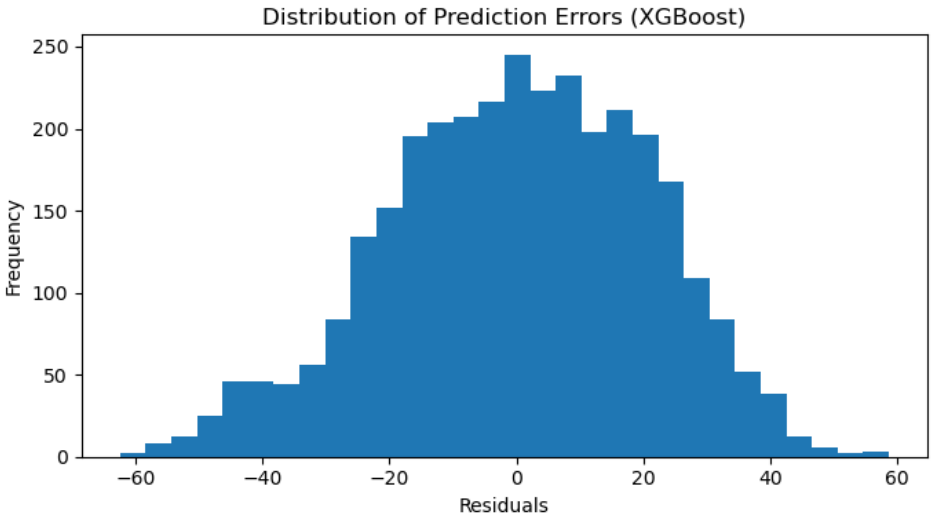


Figure 2: Distribution of Prediction Errors (XGBoost)

5 Discussion

The results reveal only modest performance differences across the evaluated models, highlighting the challenging nature of predicting Spotify track popularity. While tree-based approaches outperform linear models, the magnitude of the improvement remains limited. As shown by the performance metrics and the actual versus predicted plots, Random Forest and XGBoost capture slightly more structure in the data than Linear and Ridge Regression. XGBoost achieves the highest R^2 and the lowest error metrics. However, the absolute values remain low, indicating that even the best-performing model explains only a small fraction of the variability in popularity.

A key observation is the identical performance of Linear and Ridge Regression. Since Ridge Regression, which explicitly addresses overfitting and coefficient instability, does not improve performance relative to Linear Regression, this suggests that overfitting is not the primary limitation in this setting. Instead, the restriction lies in the linear structure of these models, which is unable to capture the non-linear relationships present in the data.

One of the main challenges observed is the difficulty in predicting extreme popularity values. The actual versus predicted plots show that linear models strongly regress predictions toward the mean, producing similar predicted values across a wide range of actual popularity scores. This behavior reflects high bias and limited model flexibility. Tree-based models partially mitigate this effect by allowing greater dispersion in predictions, particularly at higher popularity levels, but they do not eliminate it entirely. The residual distribution for XGBoost is approximately centered around zero, indicating limited systematic bias. However, its wide spread confirms that large prediction errors remain common.

Predictions cluster around mid-range popularity values because the models lack information that explains why some songs become extreme outliers. Audio features alone are insufficient to identify tracks that achieve very high popularity, which is often driven by external and contextual factors. Consequently, highly popular songs are consistently underestimated, a pattern clearly observed in the prediction plots.

These findings are broadly consistent with expectations. Given the complex and non-linear nature of music popularity, it was anticipated that linear models would struggle. The stronger performance of Random Forest and XGBoost aligns with this expectation, as these models can capture non-linear interactions between audio features. However, the relatively small improvement in R^2 suggests that non-linearity alone is insufficient to overcome the fundamental limitations of the available feature set. The fact that more flexible models provide only marginal gains

further indicates that model capacity is not the primary bottleneck, rather, the informational content of the features themselves constrains predictive performance.

The interpretation of the R^2 values in this context requires caution. Tracks with similar audio characteristics can achieve very different popularity levels due to external and unobserved factors. As a result, even well-specified models are limited in the proportion of variance they can explain. In this setting, relatively low R^2 values are expected and reflect uncertainty in the target variable rather than a failure to capture meaningful structure in the data.

An additional source of difficulty stems from the nature of Spotify's popularity metric itself. The target variable corresponds to Spotify's Popularity Index, a relative score that reflects recent engagement rather than absolute consumption. This metric aggregates multiple signals such as streams, saves, playlist additions, skip rates, and audience growth, and is used internally by Spotify to guide algorithmic recommendations. Because the exact computation is undisclosed and the score evolves dynamically over time, it introduces ambiguity into the prediction task. As a result, tracks with similar audio characteristics may receive very different popularity scores due to external influences that are not observable in the dataset.

Overall, the modest predictive accuracy highlights important limitations of the approach. Track popularity depends on many factors beyond audio content, including marketing exposure, playlist placement, artist reputation, and social dynamics. Moreover, popularity is time-dependent, whereas the models treat it as a static outcome. These constraints impose a ceiling on achievable performance and help explain why improvements across different modeling techniques remain limited.

6 Conclusion and Future Work

6.1 Summary

This project examined the ability of machine learning models to predict Spotify track popularity using audio features. Linear Regression and Ridge Regression were used as baseline models, while Random Forest and XGBoost were applied to capture non-linear relationships. The results show that tree-based models outperform linear approaches. However, the overall improvement in predictive accuracy remains modest. Even the best-performing model, XGBoost, achieves a relatively low R^2, indicating that only a limited share of the variability in popularity can be explained by audio features alone. Overall, these results highlight both the potential and the limitations of content-based approaches for popularity prediction. In particular, they show that when the target variable is complex and influenced by many external factors, predictive performance is strongly constrained by the choice of features and the nature of the popularity measure itself.

6.2 Future Directions

Several directions could be explored to improve predictive performance and extend this work. A first important improvement concerns the temporal nature of popularity. In this project, popularity is treated as a static outcome, even though it evolves over time. Modeling popularity as a time-dependent process, for instance by predicting future popularity based on early engagement signals, could better reflect real-world dynamics and reduce noise related to short-term fluctuations. However, such an approach would require repeated access to engagement data over time. Given the Spotify API rate limitation of approximately 5'000 requests per day, collecting sufficiently detailed time-series data was not feasible within the scope of this project.

A second direction for future work relates to the choice of the target variable. Spotify's Popularity Index is a relative and dynamic metric whose exact computation is undisclosed, which introduces ambiguity and noise into the prediction task. Future work could instead focus on more direct and stable measures of popularity, such as cumulative stream counts. Previous

studies, including Nijkamp (2018), suggest that modeling absolute consumption metrics may provide a clearer signal and lead to more interpretable results.

Another promising extension involves incorporating external, non-audio features. Music popularity is strongly influenced by factors beyond the intrinsic characteristics of a track, such as social media exposure, playlist placement, marketing activity, or artist visibility. Integrating external signals, for example from social media engagement or playlist metadata, could significantly improve predictive performance by capturing demand-side dynamics that audio features alone cannot explain. Combining audio-based features with such contextual information therefore represents a natural and potentially impactful direction for future research.

Additional analyses could also improve understanding of model behavior. In this project, model performance is evaluated using aggregate metrics, which may disguise differences across popularity levels. For instance, models may perform reasonably well for moderately popular songs while performing poorly for very unpopular or highly popular tracks. Examining prediction errors separately across different popularity ranges could provide more detailed insight into these limitations and help identify specific cases where model performance deteriorates.

Overall, these future directions suggest that meaningful improvements in popularity prediction will require moving beyond static, audio-only modeling toward richer temporal and contextual representations of music consumption, while accounting for practical constraints related to data availability.

References

1. Berger, W. (2017). *Why is this song popular (feat. Spotify).* Medium, https://medium.com/@albert.w.berger/what-makes-a-song-popular-in-a-certain-country-feat-pÌűiÌűtÌűbÌűuÌűlÌűlÌű-spotify-cd705abc59

2. Chen, T. et al. (2026). *xgboost: Extreme Gradient Boosting.* R package version 3.2.0.0. GitHub repository. https://github.com/dmlc/xgboost

3. Essa, Y.,Usman, A., Garg, T., Sing, M.K.(2022).*Predicting the Song Popularity Using Machine Learning Algorithm.* International Journal of Scientific Research & Engineering Trends. Volume 8 (Issue 2).

4. Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020). *Array programming with NumPy.* Nature 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2.

5. Kaggle (n.d.). *30000 Spotify Songs.* Kaggle. Accessed January 2026. https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs/data

6. Kah, Y.Y., Raheem, M.(2024), *Hit songs prediction: A review on machine learning perspective.* AIP Conference Proceedings. Volume 2802 (Issue 1). https://doi.org/10.1063/5.0183123

7. Martín-Gutiérrez, D., Hernández Peñaloza, G., Belmonte-Hernández, A. & Álvarez García, F. (2020). *A Multimodal End-to-End Deep Learning Architecture for Music Popularity Prediction.*IEEE Access. Volume 8, pp. 39361-39374. 10.1109/ACCESS.2020.2976033

8. McVicar, M. (2011). *Hit Song Science Once Again a Science.* https://api.semanticscholar.org/CorpusID:17320348

9. Pachet, F. & Roy, P. (2008). *Hit Song Science Is Not Yet a Science.* International Society for Music Information Retrieval Conference, pp. 355-360. https://doi.org/10.5281/ZENODO.1417723

10. Pandas Development Team (2020). *Pandas.* Zenodo. https://doi.org/10.5281/zenodo.3509134

11. Pedregosa, F. et al. (2011), *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825–2830.

12. Scheidegger, S. (2025) *Data Science and Advanced Programming 2025 : Lecture Notes.* Université de Lausanne

13. Smirnova, A. (2025) *Data Science and Advanced Programming 2025 : TA Practice Slides*

14. Spotify AB. (n.d.). *Get track.* Spotify for Developers. Retrieved March 10, 2025, from https://developer.spotify.com/documentation/web-api/reference/get-track

15. Spotipy Developers (n.d.). *Spotipy.* GitHub repository. Accessed January 2026. https://github.com/spotipy-dev/spotipy

16. Waskom, M.L. (2021), *Seaborn: statistical data visualization.* Journal of Open Source Software, Volume 6, p.3021 https://doi.org/10.21105/joss.03021

A Additional Figures

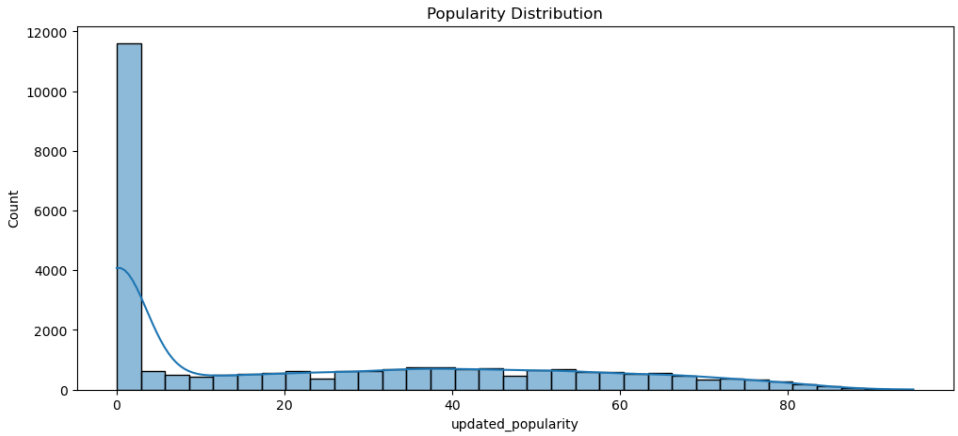


Figure 3: Distribution of Spotify popularity scores before excluding tracks with zero popularity

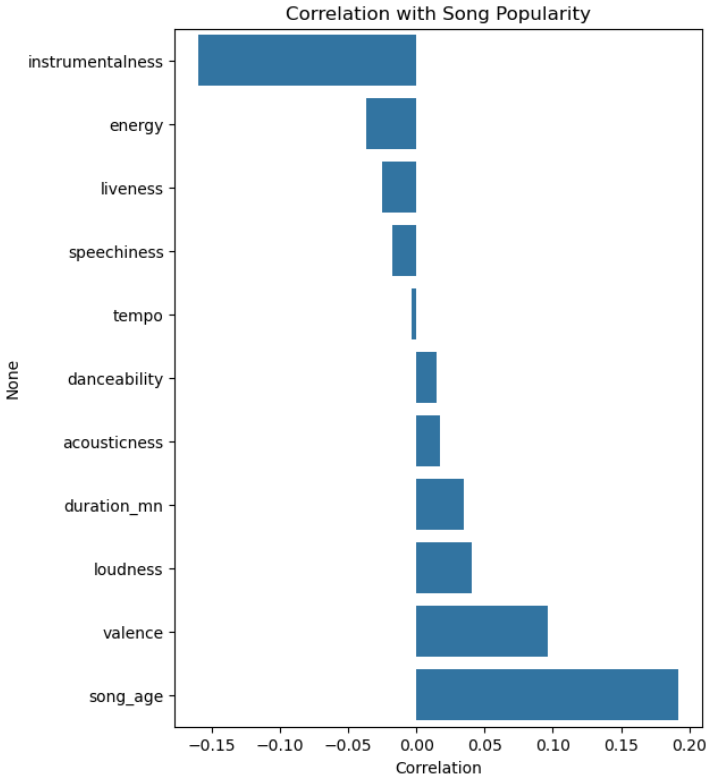


Figure 4: Correlation coefficients between audio features and track popularity

B Helper tools

The following tools were used during the preparation of this project:

- **ChatGPT (OpenAI)**: used as support for improving text clarity and resolving Python errors.
- **GitHub Copilot (integrated in Visual Studio Code)**: used for code autocompletion. All analysis and conclusions are the author's own work.

C Code Repository

GitHub Repository: `https://github.com/cleemmmence/spotify-reco.git`

- The main entry point of the project is `main.py`. The `src/` directory contains the core modules for data loading and preprocessing (`dataloader.py`), model training (`models.py`), evaluation (`evaluation.py`), and visualization (`plots.py`). The `data/` directory stores the raw and processed datasets, while the `results/` directory contains all generated figures and evaluation outputs. This report is also included in the repository.
- The project is implemented in Python (version 3.10 or higher). Dependencies are specified in the `environment.yml` file and can be installed by creating the `song-popularity` conda environment. Using this dedicated environment ensures consistency across executions.
- After activating the `song-popularity` environment, the full analysis can be reproduced by running `main.py`. This script executes the complete pipeline, including data preprocessing, model training, evaluation, and result generation. All reported performance metrics and figures are automatically saved in the `results/` directory.